Strings, Wrapper Classes & Built-in Classes

1. String Immutability and String Pool

String Immutability

In Java, a String is immutable, which means once a String object is created, its value cannot be changed.

If we modify a String, Java creates a new String object.

Example:

```
class Main {
   public static void main(String[] args) {
      String s = "Hello";
      s.concat(" Java");


      System.out.println(s);
   }
}
```

Output:

Hello

To store the changed value:

```
s = s.concat(" Java");
System.out.println(s);
```

Output:

Hello Java

String Pool

String Pool is a special memory area inside the heap where Java stores string literals to reuse them.

```
String s1 = "Java";
String s2 = "Java";
```

System.out.println(s1 == s2);

Output: true

Both variables refer to the same pooled String object.

Advantages:

- Saves memory.
- Improves performance.
- Avoids duplicate String objects.

2. Literal vs new String()

Java provides two ways to create String objects.

1. Using String Literal

String s1 = "Hello";

String s2 = "Hello";

Both refer to the same object in the String Pool.

2. Using new Keyword

String s1 = new String("Hello");

String s2 = new String("Hello");

System.out.println(s1 == s2);

Output: false

The new keyword creates separate String objects.

String Literal	new String()
Uses String Pool	Creates a new object
Memory efficient	Uses extra memory
Reuses existing literals	Creates separate objects

3. == vs equals()

Both are used to compare values or objects, but they work differently.

- == compares object references (whether both refer to the same object).
- equals() compares object content when the class provides a suitable implementation.

Example:

```
class Main {
   public static void main(String[] args) {
      String a = new String("Java");
      String b = new String("Java");

      System.out.println(a == b);
      System.out.println(a.equals(b));
   }
}
```

Output:

```
false
true
```

==	equals()
Compares references	Compares content for String
Operator	Method
May return false for equal text	Returns true for equal text

4. String Methods and Concatenation

Java provides many built-in methods to work with Strings.

Method	Use
length()	Returns string length
charAt()	Returns character at index

substring()	Extracts part of String
toUpperCase()	Converts to uppercase
toLowerCase()	Converts to lowercase
equals()	Compares content
equalsIgnoreCase()	Compares ignoring case
contains()	Checks whether text exists
replace()	Replaces characters or text
trim()	Removes leading and trailing spaces
indexOf()	Finds position of text
isEmpty()	Checks whether String is empty

Example:

```
class Main {
   public static void main(String[] args) {
      String s = "Hello Java";

      System.out.println(s.length());
      System.out.println(s.toUpperCase());
      System.out.println(s.charAt(0));
      System.out.println(s.substring(0, 5));
      System.out.println(s.contains("Java"));
   }
}
```

Output:

```
10
HELLO JAVA
H
```

Hello

true

String Concatenation

Concatenation means joining two or more Strings.

Using + operator:

String a = "Hello";

String b = "Java";

System.out.println(a + " " + b);

Using concat():

String a = "Hello";

String b = "World";

System.out.println(a.concat(b));

Output: HelloWorld

5. StringBuilder vs StringBuffer

Both classes are used to create mutable strings.

Mutable means their contents can be changed without creating a new String object for every modification.

StringBuilder:

- Faster in normal single-threaded operations.
- Not synchronized.
- Not thread-safe by default.

StringBuffer:

- Synchronized methods.
- Thread-safe for its individual method operations.
- Usually slower than StringBuilder.

Example:

```
class Main {
   public static void main(String[] args) {
      StringBuilder sb = new StringBuilder("Hello");


      sb.append(" Java");


      System.out.println(sb);
   }
}
```

Output:

Hello Java

StringBuilder	StringBuffer
Not synchronized	Synchronized
Generally faster	Generally slower
Not thread-safe by default	Thread-safe for synchronized methods
Single-threaded use	Shared multi-threaded use

6. Object Class Methods

In Java, Object is the parent class of all Java classes (directly or indirectly).

It provides important methods that can be overridden or used by other classes.

Method	Description
toString()	Returns String representation of object
equals()	Compares objects
hashCode()	Returns hash code of object
getClass()	Returns runtime class information

1. toString()

It converts an object into a String representation.

```
class Student {
    public String toString() {
        return "Student Details";
    }

    public static void main(String[] args) {
        Student s = new Student();
        System.out.println(s);
    }
}
```

Output:

Student Details

2. equals()

It compares two objects. By default, Object.equals() checks reference identity. Classes like String override it to compare content.

```
String a = "Java";
String b = "Java";

System.out.println(a.equals(b));
```

Output: true

3. hashCode()

It returns an integer hash code for an object. It is commonly used in HashMap and HashSet.

```
String s = "Java";
System.out.println(s.hashCode());
```

4. getClass()

It returns the runtime class of an object.

```
String s = "Hello";
System.out.println(s.getClass().getName());
```

Output:

```
java.lang.String
```

7. Wrapper Classes

Wrapper classes convert primitive data types into objects.

Java has eight wrapper classes.

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Example:

```
class Main {
   public static void main(String[] args) {
      int a = 10;

      Integer obj = Integer.valueOf(a);

      System.out.println(obj);
   }
}
```

Output:

10

Uses of Wrapper Classes:

- Convert primitive values into objects.
- Useful in collections like ArrayList.
- Provide methods for conversion.
- Help in working with generic classes.

8. Autoboxing and Unboxing

Autoboxing

Autoboxing means automatic conversion of a primitive type into its wrapper object.

int a = 10;

Integer obj = a;

System.out.println(obj);

Output: 10

Here, int is automatically converted into Integer.

Unboxing

Unboxing means automatic conversion of a wrapper object into a primitive type.

Integer obj = 20;

int a = obj;

System.out.println(a);

Output: 20

Difference:

Autoboxing	Unboxing
Primitive to object	Object to primitive
int → Integer	Integer → int
Automatic conversion	Automatic conversion

Note: Unboxing a null wrapper reference causes NullPointerException.

9. parseXxx() and valueOf()

These methods are used to convert String values into other data types.

parseXxx()

It converts a String into a primitive value.

```
String s = "100";

int a = Integer.parseInt(s);

System.out.println(a + 50);
```

Output:

150

valueOf()

It converts a String into a wrapper object.

```
String s = "200";

Integer a = Integer.valueOf(s);

System.out.println(a);
```

Output:

200

parseXxx()	valueOf()

Returns primitive	Returns wrapper object
parseInt() returns int	valueOf() returns Integer
Useful for calculations	Useful for object-based operations

Common conversion methods:

Method	Return Type
Integer.parseInt()	int
Double.parseDouble()	double
Float.parseFloat()	float
Long.parseLong()	long
Boolean.parseBoolean()	boolean
Integer.valueOf()	Integer
Double.valueOf()	Double

10. Math, System and Runtime Classes

Java provides built-in classes for mathematical calculations, system operations and runtime information.

A. Math Class

The Math class belongs to java.lang package.

It provides mathematical methods and constants.

Method	Use
Math.abs()	Returns positive absolute value
Math.max()	Returns maximum value
Math.min()	Returns minimum value
Math.sqrt()	Finds square root
Math.pow()	Calculates power

Math.round()	Rounds a number
Math.ceil()	Rounds upward
Math.floor()	Rounds downward
Math.random()	Generates random double

Example:

```
class Main {
   public static void main(String[] args) {
      System.out.println(Math.max(10, 20));
      System.out.println(Math.sqrt(25));
      System.out.println(Math.pow(2, 3));
      System.out.println(Math.abs(-50));
   }
}
```

Output:

```
20
5.0
8.0
50
```

B. System Class

The System class provides methods to interact with the Java system.

Important methods:

Method	Use
System.out.println()	Prints output
System.in	Takes input
System.err	Displays error output

System.currentTimeMillis()	Returns current time in milliseconds
System.exit()	Terminates the program
System.gc()	Requests garbage collection
System.getProperty()	Gets system property

Example:

```
class Main {
   public static void main(String[] args) {
      System.out.println("Hello Java");


      long time = System.currentTimeMillis();


      System.out.println(time);
   }
}
```

C. Runtime Class

The Runtime class provides information about the current Java application runtime environment.

It belongs to the java.lang package.

Important methods:

Method	Use
getRuntime()	Gets Runtime object
availableProcessors()	Gets available processor count
freeMemory()	Gets free memory
totalMemory()	Gets total allocated memory
maxMemory()	Gets maximum memory limit
gc()	Requests garbage collection

exit() Terminates the JVM

Example:

```
class Main {
    public static void main(String[] args) {
        Runtime r = Runtime.getRuntime();

        System.out.println("Processors: "
                + r.availableProcessors());

        System.out.println("Free Memory: "
                + r.freeMemory());

        System.out.println("Total Memory: "
                + r.totalMemory());
    }
}
```